COMS4995W30
Applied Machine Learning

Dr. Spencer W. Luo

Columbia University | Fall 2026



LLM Pre-training & Fine-tuning

AI of the Week: Pre-training Still Matters



DeepSeek V4.1 Flash

- The model card reports 45 trillion multimodal pre-training tokens.
- Its recipe retains SFT, reinforcement learning, and distillation.

The lesson for this course

- Pre-training learns broad patterns; post-training shapes behavior.
- Different stages need different data and evaluation sets.

AI of the Week: Better Training Data



The reported change is in the data pipeline

- The team emphasizes generated tasks, environments, and rollouts.

A useful engineering question

- Which failure needs better examples, and which needs better feedback?

Discuss with a partner for two minutes

- A support model uses the wrong output format. What would you try first?



Model Family

Large Language Models (LLMs) 🤖



LLMs are:

- Transformer-based models
- Trained on massive text corpora - often trillions of tokens
- Learn general-purpose language understanding and generation abilities

[Scaling Law] Model parameters, data, and compute → emergent intelligence

Key Characteristics:

- Trained with self-supervised next-token prediction
- Exhibit few-shot / zero-shot capabilities
- Can be further adapted via fine-tuning to many downstream tasks

From Architecture → Paradigm ⚙️



Old View: “Architecture Defines Intelligence”

- Researchers focused on building smarter structures (LSTM, Transformer)
- But even elegant designs failed without scale and rich data

Modern View: “Data + Algorithm Define Capability”

- BERT, GPT, and T5 share Transformer building blocks, but differ in architecture, masking, objectives, and data

From Architecture → Paradigm ⚙️



Key Insight:

- Architecture shapes foundation, but data and algorithm shape behavior
- The training paradigm now matters more than structural novelty

Takeaway:

Modern breakthroughs (ChatGPT, Gemini) come not from new layers, but from how we teach and scale the same architecture, with better data

Analogy: Human Learning 🧍‍♀️

Stage	Human	Model (LLM)
Pre-training	Learning from reading, observation, and experience about the world	Trained on massive text corpus - learns language, facts, and world knowledge
Fine-tuning	Getting job-specific/domain training (e.g., becoming an engineer etc.)	Task-specific SFT on labeled data - adapts to domains or strengthens reasoning
Reinforcement Learning	Socialization and feedback - learning norms, politeness, ethics from others	Preference tuning (RLxF) - adjusts to ANY preference signals

Flow: Foundation → Specialization → Alignment

From Building Blocks to Model Families

The same Transformer block can act very differently depending on:

- Masking pattern – what each token can “see”
- Training objective – what the model is asked to predict

These choices define 3 major model families:

- Decoder-only → generates text step by step
- Encoder-only → understands full context
- Encoder-Decoder → translates one sequence into another

Decoder-only Language Models 💬



Mask: Causal → token t can observe only tokens ≤ t

Objective: predict the next token from the preceding context

Usage:

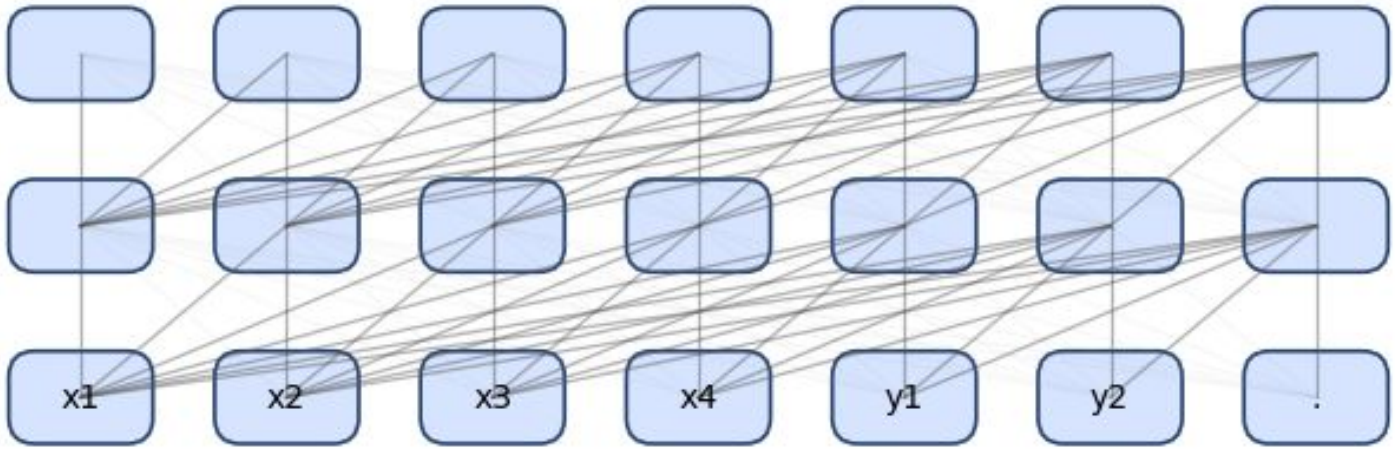
- Natural language generation (chatbots, storytelling)
- Code generation and completion (Copilot, Claude)

Examples:

GPT-2 / GPT-3 / GPT-4, LLaMA, Gemini, Mistral….

Decoder-only

causal self-attention



Encoder-only Masked Language Models 🔍



Mask: Full attention → all tokens can attend bidirectionally

Objective: Predict randomly masked tokens using both-sided context

Usage:

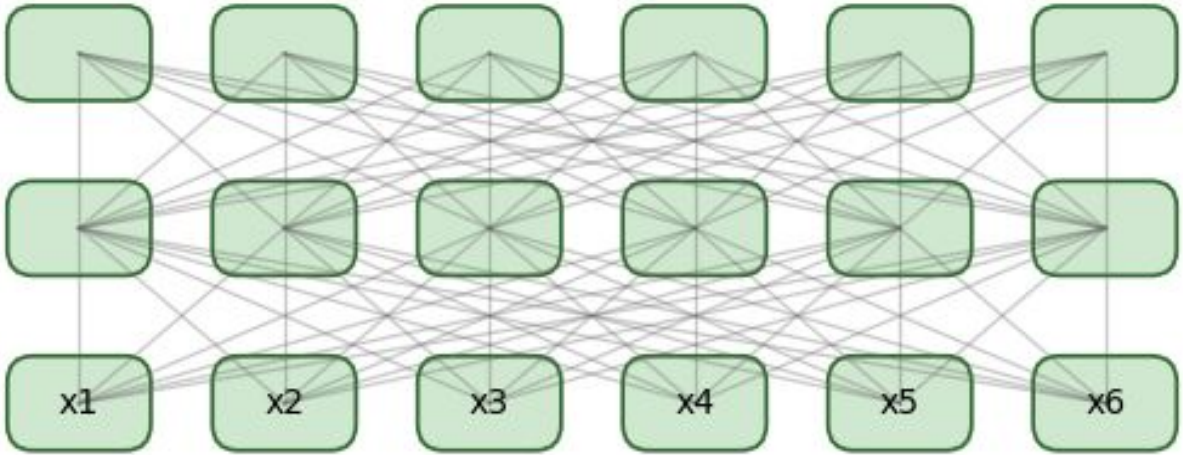
- Text classification (sentiment, topic)
- Retrieval and ranking (semantic search, dense embeddings)

Examples:

BERT, RoBERTa, SpanBERT, ALBERT……

Encoder-only

bi-directional self-attention



Encoder–Decoder Models 🔁



Mask:

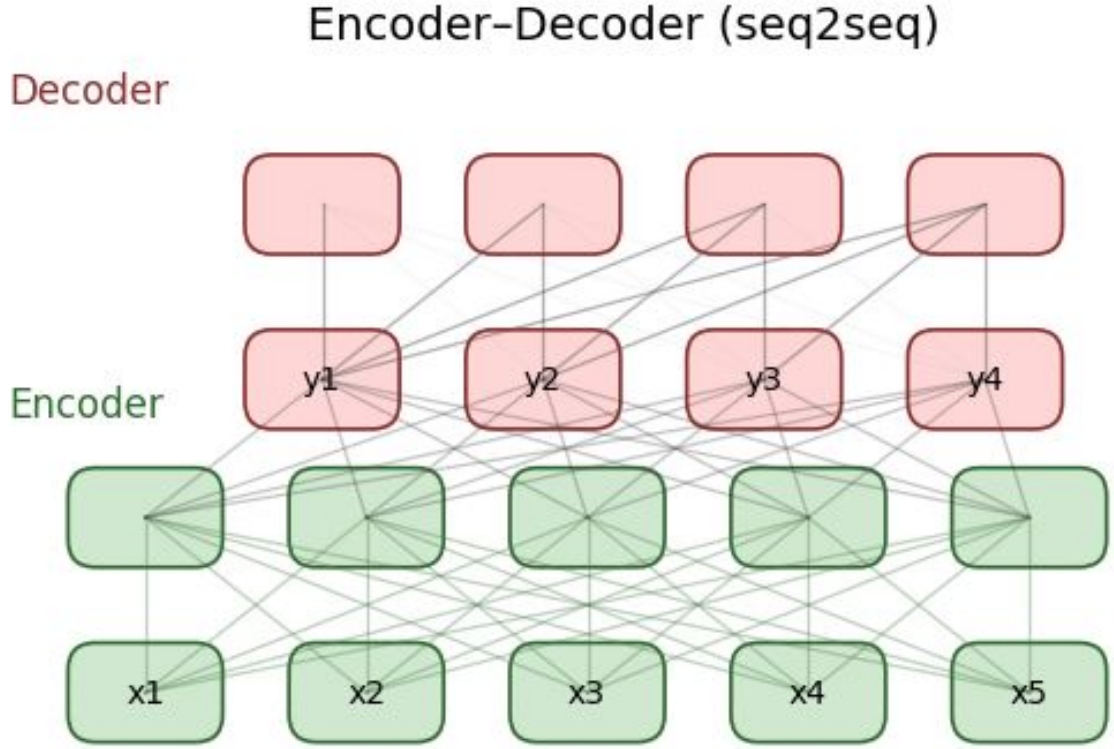
- Encoder → Full attention (understanding input)
- Decoder → Causal mask (generate token by token)
- Cross-attention → connects decoder queries to encoder outputs

Usage:

- Machine translation
- Image → text generation

Examples:

- T5, BART, mT5, MarianMT……


Encoder–Decoder (seq2seq)
Decoder
Encoder
y1
y2
y3
y4
x1
x2
x3
x4
x5

Summary



Free text corpus → Decoder-only (Language Modeling)

- No paired data, just raw text
- Train to predict the next token - learns grammar, style, and coherence

Feature learning or retrieval → Encoder-only (Masked LM)

- Focus on deep understanding, not generation
- Learns contextual embeddings for classification or search

Labeled input/output pairs → Encoder–Decoder (Seq2Seq)

- Each input has a corresponding target (translation, summarization)
- Model learns conditional mapping P(output | input)



LLM Pre-training

What Is Pre-training? 📘



Train on a massive unlabeled text corpus

- Billions of words from books, and web pages - no manual labeling needed
- The model learns patterns of syntax, and world knowledge implicitly

Self-supervised objective:

- Does not need label
- Predict masked (Encoder-only) or next tokens (Decoder-only) from context
- The task creates its own supervision signal by hiding or shifting tokens

Pre-training Examples



Example 1 - Masked Language Modeling (BERT-style)

Input:

"The cat [MASK_1] on the [MASK_2]."

Target:

"The cat sat on the mat."

👉 The model must use bidirectional context to predict the masked token

Pre-training: Predict the Next Token



Input prefix	Target	p(target)	−ln p
<BOS>	The	0.50	0.693
<BOS> The	cat	0.25	1.386
<BOS> The cat	sat	0.80	0.223

$$\text{Mean loss} = (0.693 + 1.386 + 0.223) / 3 \approx 0.768$$

What Is Pre-training? 📘



Outcome:

The model learns a general-purpose representation of language

- capturing grammar, meaning, and relationships between entities

Result:

The pre-trained model becomes:

- a reusable foundation
- easily adapted to many downstream tasks (classification, summarization, etc.) through fine-tuning

Why Self-Supervised? 🪄



Training signal comes directly from the data itself

Every word, punctuation, or phrase provides a new learning opportunity

This makes the world knowledge effectively one massive training corpus

Scales effortlessly:

Since labeling is automatic, models can train on trillions of tokens across diverse domains (books, code, social media etc)

Pre-training Data 📚



High-Quality Curated Web Data

- StackExchange / Reddit / Hacker News / ArXiv / PubMed - filtered for long-form reasoning, technical correctness, and linguistic quality
- C4 (Colossal Clean Crawled Corpus) - curated Common Crawl derivative used in T5
- RefinedWeb / Dolma / RedPajama / Falcon RefinedWeb - open-source large-scale cleaned web corpora

Pre-training Data 📚



Instructional / Human-Curated Data

- Wikipedia discussions, StackOverflow Q&A, instructional forums — naturally structured in “question → answer” form
- QA training splits: Natural Questions, SQuAD, GSM8K; keep evaluation data held out

Educational and Academic Texts

- Textbooks, lecture notes, academic papers (via S2ORC or ArXiv) - help models internalize reasoning and math syntax
- Project Gutenberg + scientific book scans - classic long-form, well-structured writing

Pre-training Data 📚



Code and Technical Content

- GitHub, StackOverflow, Jupyter Notebooks, Competitive Programming datasets

Dialogue & Conversational Corpora

- OpenSubtitles, MultiWOZ, OpenOrca - to enhance conversational fluency

Multilingual and Multimodal Text

- Multilingual datasets: Wikipedia in 100+ languages - encourage cross-lingual generalization
- Multimodal text: image captions, audio transcripts, or paired HTML/context

Pre-training Data 📚



Synthetic & Augmented Data (increasingly common)

Self-generated text from earlier LLMs (self-play, bootstrapping, or distillation)

Data augmentation via back-translation, paraphrasing, or chain-of-thought synthesis

Steps: Data cleaning → Deduplication → Quality filtering

Goals: Diversity, quality, and safety → Teach LLMs the basic knowledge

Pre-training Data Cleaning Pipeline 🧹



Deduplication

- Remove exact and near-duplicate documents using MinHash or SimHash

Content Filtering

- Exclude boilerplate (HTML templates, navigation bars, ads)

Toxicity & Safety Filtering

- Use classifiers to detect hate, harassment, adult, or violent content
- Combine rule-based filters with model-based toxicity scoring

Data Cleaning: Four Decisions

Document	Observed issue	First action	Reason
useful article	Cookie banner	Remove boilerplate	Keep its content
same article	Reposted copy	Cluster	Avoid repeats
private log	Email + secret	Exclude / redact	Protect private data
eval answer	Benchmark overlap	Quarantine	Protect the test

Toy corpus. Decisions require provenance and context; one filter is not enough.

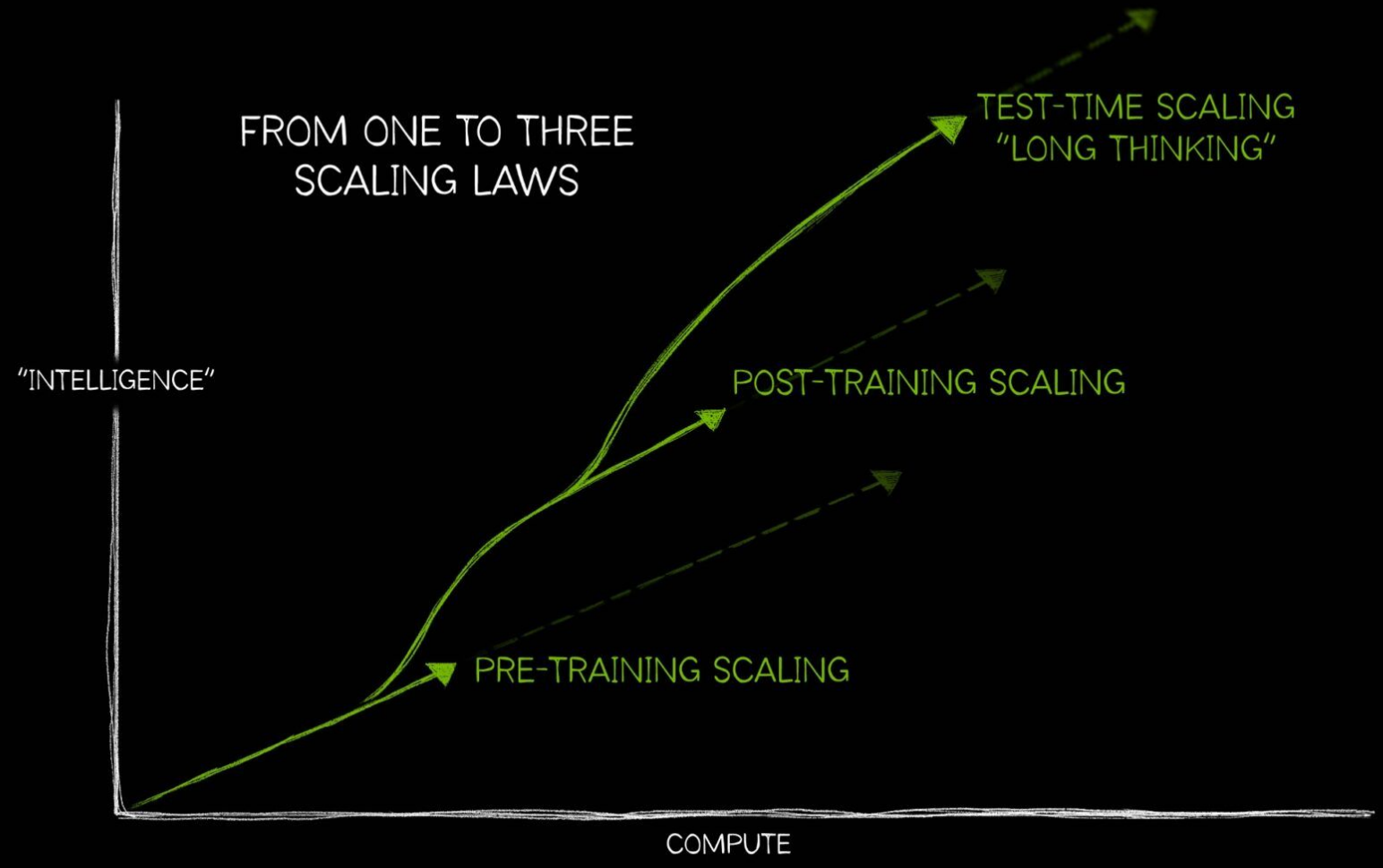
Scaling Laws 📈



Model performance improves predictably as model size, data, and compute increase - following a power-law trend

Patterns observed:

- More data → smoother loss curves, better generalization
- Larger models → capture richer structure, longer reasoning chains
- Emergent abilities appear only after certain scale thresholds
- But returns diminish - doubling data ≠ doubling capability


FROM ONE TO THREE
SCALING LAWS
TEST-TIME SCALING
"LONG THINKING"
POST-TRAINING SCALING
PRE-TRAINING SCALING
"INTELLIGENCE"
COMPUTE

Why Next-Token Prediction Matters 🤔



The next token prediction task is extremely simple

Yet applied at massive scale the model implicitly learns many tasks

Grammar: “In my free time, I like to {run, banana}” → predicting “run”

World knowledge: “The capital of Japan is {Copenhagen, Tokyo}” → predicting “Tokyo”

Sentiment: “I was laughing the entire time, the movie was {good, bad}” → predicting “good”

Reasoning & Logic: “If it rains, the ground will be {wet, dry}.” → wet

Math & Symbolic Reasoning: “2 + 3 = {5, 6}” → 5

What the Model Learns 🧠



Syntax / Semantics / Factual knowledge / Basic reasoning capabilities / ……

Each next-token prediction is like solving a micro-task

- fill in a blank, finish a phrase, complete a thought

Across billions of sentences, the model encounters millions of implicit tasks - translation, analogy, classification, reasoning, summarization

The model learns patterns but can also memorize training text

- learning which continuations are plausible, logical, and coherent

Emergent capability 🌍



At sufficient scale, next-token prediction yields general intelligence-like behavior

- models can perform tasks they were never explicitly trained for, by transfer what they have learned from languages

This phenomenon is often summarized as:

- Pre-training teaches basic knowledge
- Supervised Fine-tuning enhance reasoning capabilities
- Reinforcement Learning aligns it

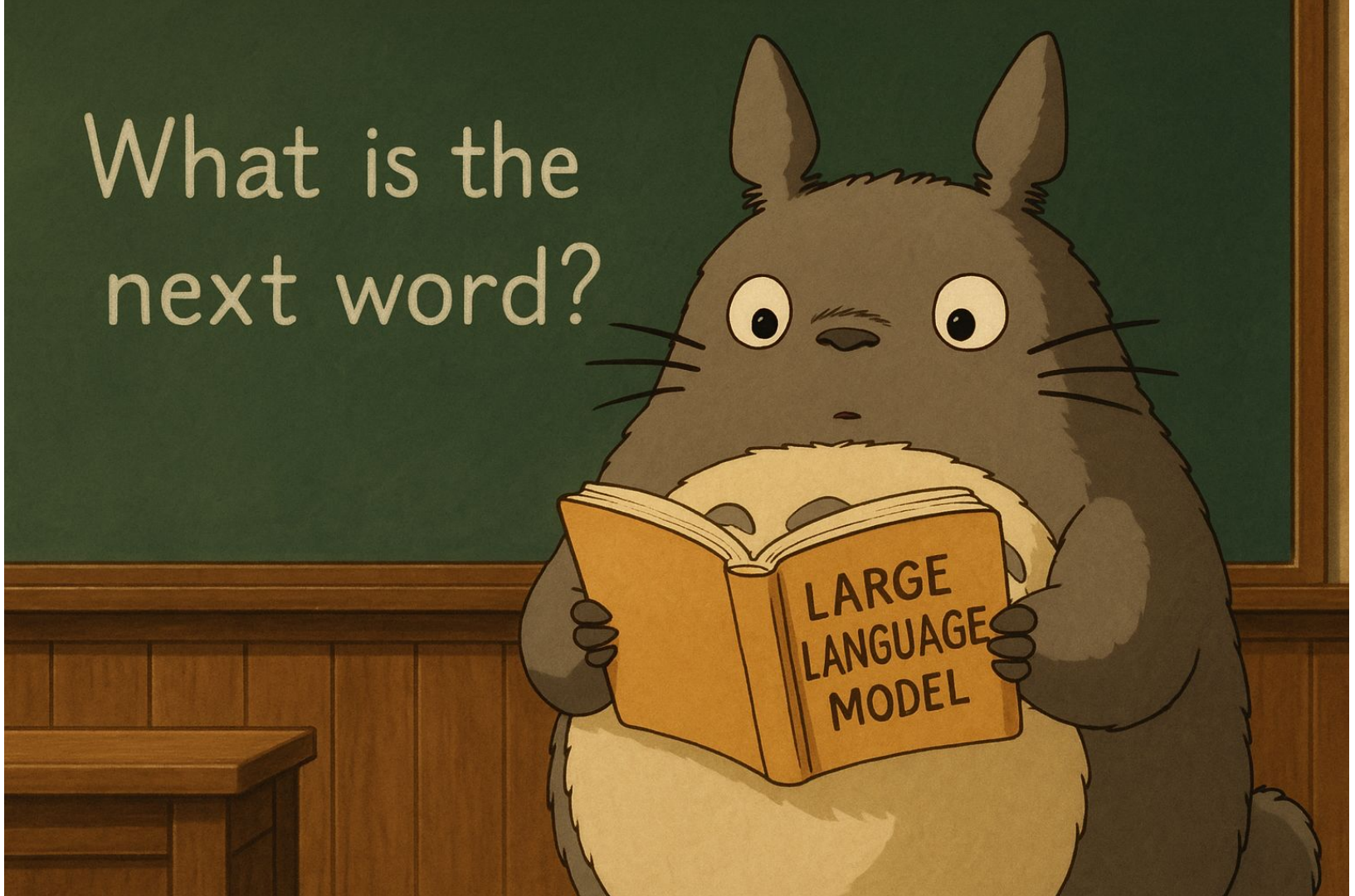
General Artificial Intelligence Step 0 🧠



Transformer + next-token objective + large data

→ General-purpose language model

Emergent abilities appear once enough scale + data + training


What is the next word?
LARGE LANGUAGE MODEL



LLM Supervised Fine-tuning

From Pre-training to Fine-tuning 🌉

Pre-training: Learns general world knowledge from massive unlabeled text

- Models know how to speak but not what to do

Fine-tuning: Teaches behavior and task goals using labeled examples

- Bridge: Moves from “foundation knowledge” → “actionable skills”

Supervised Fine-tuning (SFT) 🧭



Train a pre-trained LLM on (prompt, response) pairs for a target task/behavior

Goal: Align the model to follow instructions and produce useful, reliable outputs

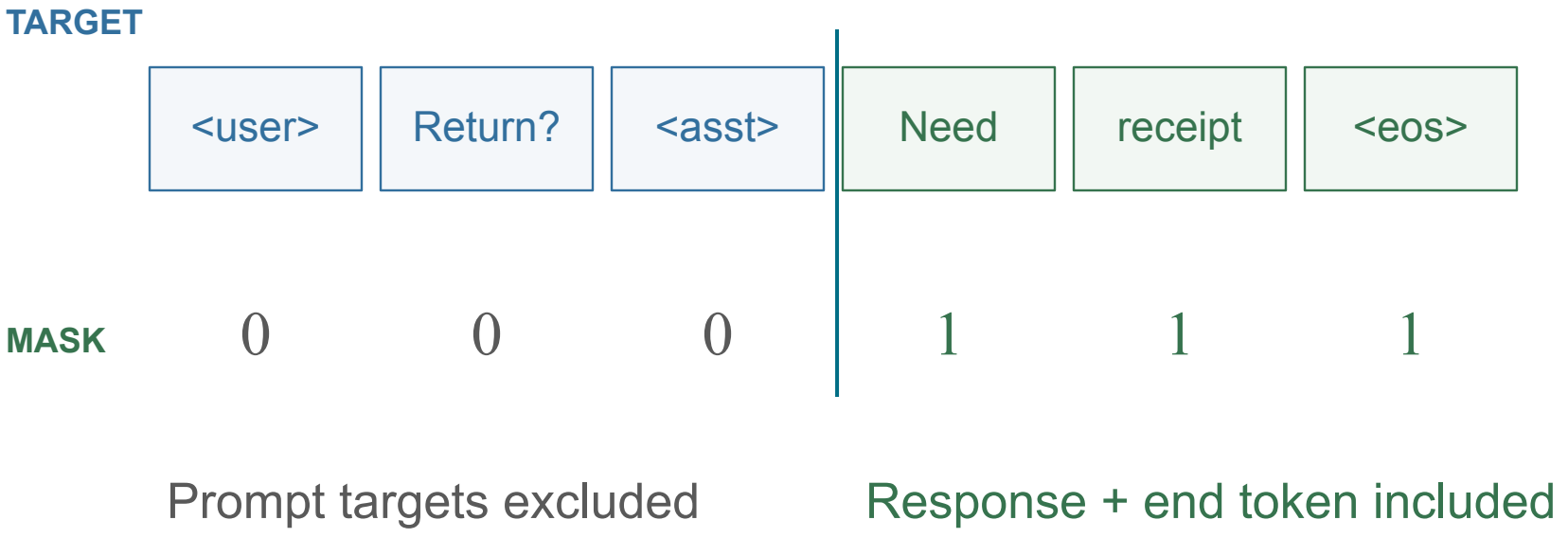
Setup:

- Update all weights (full SFT), or train adapters with a frozen base (PEFT)
- Minimize token-level cross-entropy on target responses

Data: High-quality, diverse instructions with clear, verifiable targets

SFT: Which Tokens Count?

A toy target sequence, after the one-position shift:



Standard SFT Paradigm

Example:

Prompt: “Summarize the paragraph.”

Response: “Climate change is harming coral reefs.”

Model learns to:

- Follow instructions
- Generate fluent, polite, and relevant text

Limitation: imitates supplied targets; does not independently verify them

Why Modern LLMs Are “Thinking Models” 

From Pattern Matching → Structured Reasoning

- Early LLMs mimicked language fluency
- Newer models learn to analyze, plan, and reason across multiple steps

Training Evolution

- Pre-training on vast, diverse data teaches latent world models
- SFT and RL explicitly train reasoning traces (“think step by step”)

Behavior Shift

- Models now explain reasons before answering
- Reasoning traces can help, but plausible steps can still be wrong

Thinking SFT is Step 1



Standard SFT = aligns immediate behavior (instruction-following)

Thinking SFT = aligns reasoning (how the model reaches conclusions)

Without reasoning signals, models learn to “answer fast” instead of “think slow”

Goal: expose the model to step-by-step problem solving, not just end results

💡 We are teaching the model to show its work - not just the answer

What Are “Thinking Tasks”?



Tasks where intermediate reasoning is essential, not optional

Example 1 - Math / Logic

Prompt: “What is 27 × 14?”

Reasoning: “27 × 10 = 270, 27 × 4 = 108, 270 + 108 = 378.”

Answer: “378.”

Example 2 - Everyday Reasoning

Prompt: “Why do we wear jackets in winter?”

Reasoning: “Because jackets trap body heat, keep us warm when the air is cold.”

Answer: “To stay warm.”

Example 1 – Plain SFT 🧱



Prompt: “Solve: 12 + 7 = ?”

Response: “19.”

Loss covers response tokens only; exact tokenization varies

→ The model learns to produce correct answers and follow format

Intermediate reasoning steps are not explicitly supervised here

Example 2 – Reasoning-augmented SFT



Prompt: “Solve: 12 + 7 = ?”

Response: “Let’s think step by step. 12 + 7 = 19. Answer: 19.”

Now the loss covers the rationale and the final answer

Only response targets contribute to the loss.

→ The model learns that generating intermediate reasoning steps reduces the overall loss

→ The behavior “explain before answering” becomes part of training

Where to Source Thinking Data ⚙️



Math / Logic: GSM8K, MATH, AQUA-RAT, SVAMP

Reasoning QA: StrategyQA, HotpotQA, OpenBookQA

Step-by-step: CoT (Chain-of-Thought) traces from Flan, CotHub, OpenOrca

Code Reasoning: CodeContests, LeetCode-HF, DeepMind MathCode datasets

Human-curated: Collect instructor-written rationales or worked solutions

Synthetic Generation: Use an existing LLM to produce reasoning traces

(“Let’s think step by step…” → label + verify)

Filter by correctness or self-consistency

How to Curate and Format 📚



Keep structure consistent

- Prompt: [problem]
- Reasoning: [chain of thought]
- Answer: [final result]

Keep reasoning concise

- 2–6 reasoning steps preferred
- Each step should be verifiable (not vague “I think maybe...”)

How to Curate and Format 📚



Keep correctness

- Human or model verification for every rationale
- Discard hallucinated or logically invalid traces

Keep diversity

- Mix numeric, symbolic, commonsense, procedural reasoning
- Include both short and long-context tasks

Beyond Single-task SFT 🌐



Multi-task fine-tuning: Train on many instruction types simultaneously

Mixture of formats: QA + translation + summarization + more → boosts skills

Instruction format standardization: Convert all tasks to prompt-response pairs

Results: Better zero-shot transfer and alignment readiness

Industry trends

Looking Ahead



The pre-training + fine-tuning paradigm keeps evolving rapidly

New research improves efficiency, scalability, and capability

Understanding these trends helps you stay ahead of the LLM curve

Focus today:

- Efficiency
- Multimodality
- Domain adaptation

Parameter-efficient Fine-tuning (PEFT) ⚙️



Challenge: Full fine-tuning = billions of parameters → expensive & slow

Idea: Freeze the base model; only train small additional modules

Goal: Reduce compute + memory footprint while keeping performance

Result: Same model reused across many downstream tasks efficiently

LoRA: Low-Rank Adaptation 💾



Key idea: Instead of updating full weight matrix W, learn 2 small matrices A & B s.t.

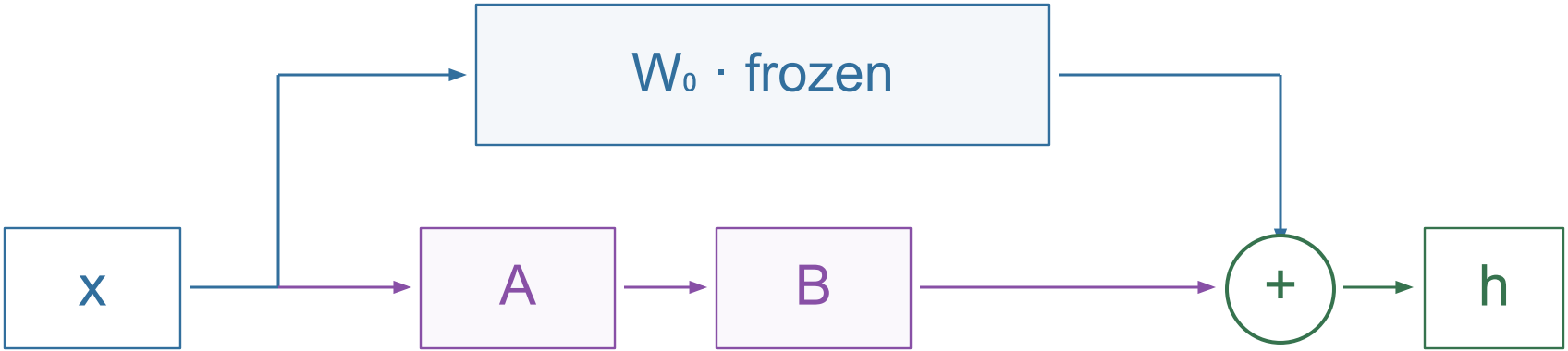
$$\Delta W = (\alpha/r)\, BA \text{ (low-rank update)}$$

Benefits:

- A small trainable fraction, depending on rank and target layers
- Easy to merge/unmerge with base model
- Great for multi-task learning and low-data scenarios

LoRA: One Layer, Two Paths

A residual branch adds a learned correction.



Trainable branch · scale by α/r before adding

LoRA: Count the Trainable Parameters

One 4096 × 4096 projection; choose rank r = 8.

FULL MATRIX

16,777,216

4096×4096

LoRA A + B

65,536

$8 \times (4096 + 4096)$

About 0.39% as many trainable parameters.

QLoRA: Quantized LoRA ⚡



Keep the base model frozen and store its weights in 4-bit precision.

Train LoRA adapters in higher precision.

QLoRA (2023): a 65B model fine-tuned on one 48GB GPU.

Useful when GPU memory is limited; task quality still needs evaluation.

🎤 Mock Interview

You have 8,000 labeled support examples and one GPU. The base model is useful, but its responses need a consistent format.

A) Pre-train a new model from scratch
B) Use LoRA after testing a prompt baseline
C) Train on the evaluation answers too
D) Increase the context window only

2 minutes with a partner. Choose one answer and explain why.

Summary



Pre-training: foundation of general intelligence

SFT: learns instruction-following and task behavior from examples

Emerging trends: efficiency and multimodality

Mindset shift: Foundation models are not endpoints - but platforms